RBSRN: Role-Aware and Background-Conditioned Spatiotemporal Residual Network for Multi-Step Regional Traffic Flow Forecasting

Abstract

Short-term regional flow forecasting must predict both inflow and outflow over several future intervals while traffic conditions change. We propose the Role-aware and Background-conditioned Spatiotemporal Residual Network (RBSRN), which first produces a shared base prediction and then adds two parallel corrections. The State-Evidence Expert Residual Refinement Module (SERR) combines self-evolution and directed time-lagged experts according to the current state. Background-conditioned Cross-region Message Selection (BCMS) selects neighbor–lag–scale messages for each forecast step using source and target traffic backgrounds. On four Chengdu and Shanghai forecasting tasks, RBSRN achieved the lowest 12-step average MAE under positive-target evaluation, with reductions of 0.43%–2.00% relative to the strongest baseline on each task. Fixed-model residual-removal diagnostics further showed that, at the 30-minute horizon, self-evolution correction produced 1.15–2.82 times larger MAE benefits for rapidly changing histories than for slowly changing histories. BCMS produced its largest mean benefit for the group with the smallest history-derived background-discrepancy proxy across all 12 horizons. These results link residual correction to the current traffic state and background.

Keywords: regional traffic flow forecasting; multi-step forecasting; spatiotemporal residual learning; role-aware gating; background-conditioned historical message selection

1 Introduction

Forecasts of regional travel demand guide fleet dispatch and the allocation of transport resources [1]. Regional inflow and outflow count trips entering and leaving each region within a time interval [2]; their temporal and spatial variation determines where demand concentrates and how long it persists. Dispatch decisions therefore require joint multi-step forecasts of both flows over the next 5–60 minutes [3], [4]. Traffic event information can supplement historical observations in short-term flow forecasting [5], but a useful model must also distinguish routine spatiotemporal variation from local departures from that routine.

Existing forecasting models improve performance by learning spatial dependencies, temporal patterns, and multiscale dynamics. ST-ResNet models temporal closeness, periodicity, and trends with residual convolutions; on the single-step TaxiBJ task at 30-minute resolution, its best configuration reduced RMSE from 18.18 for DeepST to 16.69 [6]. Graph-based models learn spatial dependencies through diffusion convolution, graph convolution, or adaptive adjacency matrices [7], [8], [9]. STWave and TEDDCF separate low-frequency trends from high-frequency dynamics with two processing branches [10], [11], while ST-GDN combines local geographic dependencies, global regional associations, and multiscale temporal attention [12]. These methods expand the historical evidence available to the predictor. The value of this evidence changes when local flow departs from routine patterns and when the forecast horizon

changes. The remaining problem is how to turn heterogeneous historical evidence into corrections that fit the current state and each future step.

The sources of evidence also play different roles. A node's own history records local continuation and turning points, whereas other regions provide cross-region references through directed time-lagged associations [13]. Their spatial origins and temporal alignment differ, and the relevance of a message depends on the source and target backgrounds as well as the forecast horizon. Figure 1 illustrates this point with outflow from Chengdu grid cell 601 in November 2016. Two recent states yielded historical correlations of 0.859 and 0.907 with a lag-aligned neighboring region, but the mean absolute differences between their subsequent 30–60-minute trajectories were 0.5 and 6.0 counts per 15 min. Similar historical association therefore led to different future deviations. A forecasting model should assign self-evolution and cross-region evidence distinct correction roles and select historical messages separately for each future step.

Recent models address heterogeneous dynamics and cross-region information through signal disentanglement, expert routing, and delay-aware modeling. D2STGNN separates diffusion and inherent signals with gating and residual decomposition [14]; TESTAM routes inputs among temporal, static-graph, and dynamic-graph experts [15]; and PDFormer uses delay-aware feature transformations to handle temporal misalignment in cross-region information [16]. These designs distinguish sources of information, but they leave open how each source should correct a shared prediction when traffic departs from routine behavior. This paper addresses that question by relating correction roles to the current state and cross-region message selection to background discrepancy and forecast horizon.

We propose the Role-aware and Background-conditioned Spatiotemporal Residual Network (RBSRN). RBSRN first encodes low- and high-frequency traffic patterns to produce a shared base prediction, then adds two parallel corrections. The State-Evidence Expert Residual Refinement Module (SERR) assigns separate roles to self-evolution and directed time-lagged experts and gates their contributions using the current state. Background-conditioned Cross-region Message Selection (BCMS) uses source and target low-frequency backgrounds to select high-frequency neighbor–lag–scale messages for each forecast step. The two modules read the shared representations in parallel and add their corrections at the prediction stage.

The main contributions are as follows:

1. SERR separates local self-evolution from directed time-lagged evidence and gates their residual corrections with the current traffic state. On the Chengdu ablation, removing either expert increased MAE and RMSE for both inflow and outflow.

2. BCMS selects neighbor–lag–scale messages jointly from source and target backgrounds for each forecast step. Fixed-model residual-removal diagnostics in both cities found larger mean MAE benefits when the history-derived background-discrepancy proxy was smaller.

3. Across Chengdu and Shanghai, RBSRN achieved the lowest AVG12 MAE and RMSE on all four city–channel tasks against 13 baselines under positive-target evaluation. Its MAE reduction relative to the strongest baseline was 0.43%–2.00%.

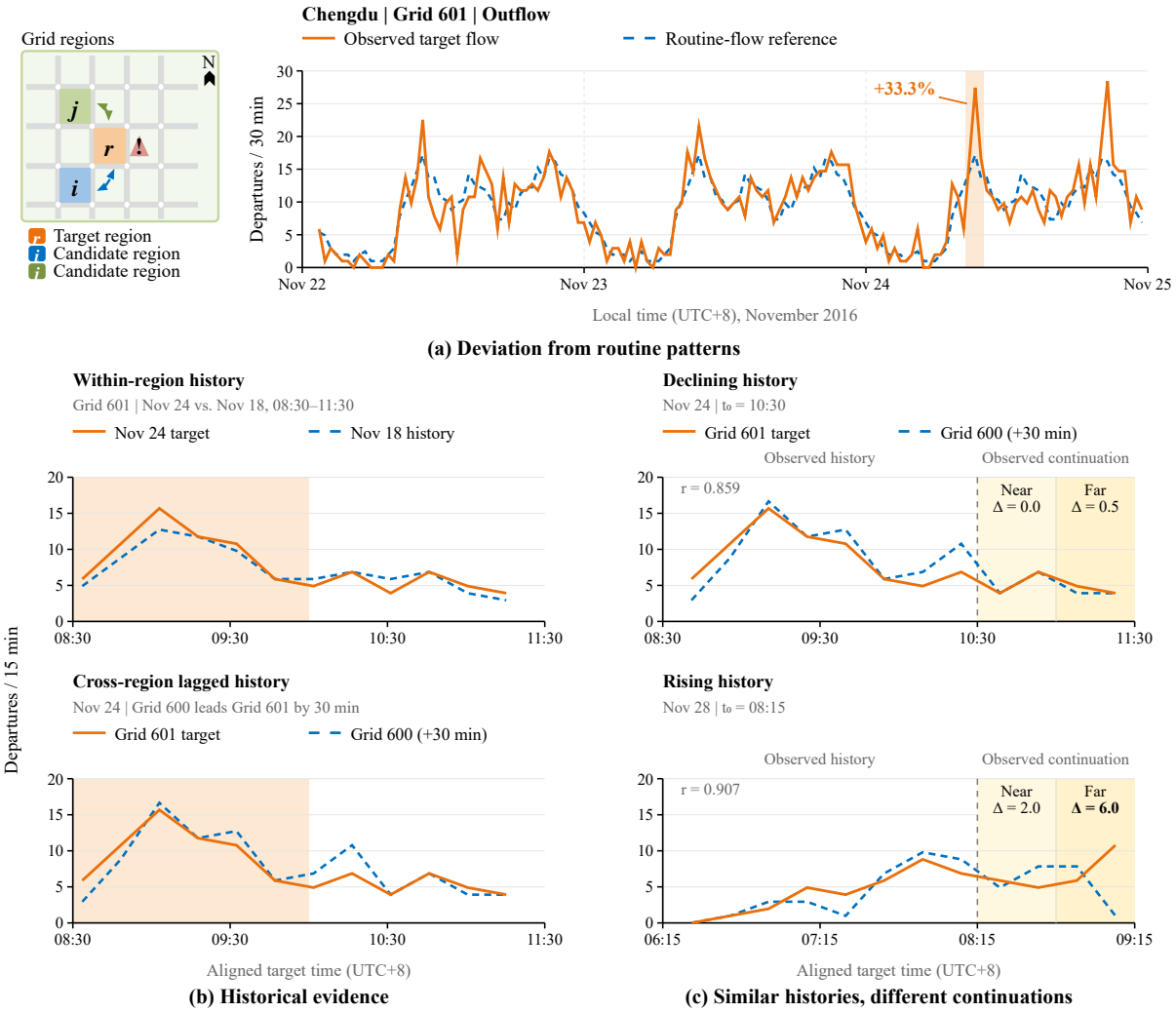


Figure 1 Observed deviations from routine flow and cross-region references. (a) Routine and observed flow. (b) Within-region and cross-region historical evidence. (c) Divergent future trajectories.

2 Related Work

2.1 Spatiotemporal Representations and Temporal Pattern Disentanglement

Traffic flow forecasting combines spatial dependency modeling with temporal encoding. Graph neural networks make this structure explicit by modeling associations between traffic nodes [17]. DCRNN represents propagation as diffusion on a directed graph and combines diffusion convolution with recurrent units [7]; STGCN couples graph and temporal convolution [8]; and Graph WaveNet learns latent relations with an adaptive adjacency matrix and dilated convolutions [9]. STID adds spatial and temporal identity embeddings to distinguish nodes and time slots [18]. These models define the standard spatiotemporal representation used by regional forecasters.

Attention and adaptive relation learning further refine this representation. ASTGCN and GMAN use spatial and temporal attention to focus on relevant locations and times [19], [20]. MTGNN and AGCRN learn node relations from data [21], [22], while STSGCN and STFGNN

construct local spatiotemporal graphs from node–time associations and sequence similarity [23], [24]. These models improve dependency learning while maintaining a unified representation of the traffic history.

Temporal pattern disentanglement organizes encoding around frequency or periodic structure. STWave uses a discrete wavelet transform to separate low- and high-frequency components [10]; TEDDCF separates trend and event components and assigns them different spatial and temporal paths [11]; and other methods distinguish periodic from aperiodic variation [25]. HyperD represents periodic patterns with daily and weekly embeddings, models residual fluctuations in the frequency domain, and aligns the two views [26]. These approaches make the base representation more responsive to dynamics at different scales.

These methods strengthen the base representation. They leave a complementary question: how should information from different sources correct the prediction when traffic departs from routine behavior?

2.2 Heterogeneous Evidence and Expert Modeling

Traffic dynamics vary across regions and time periods. ST-SSL uses spatial and temporal self-supervised tasks to strengthen representations of this heterogeneity [27]. ST-ResNet uses three residual branches for temporal closeness, periodicity, and trends, and learns to fuse their outputs [6].

Other methods explicitly distinguish traffic dynamics through disentanglement or expert mechanisms. D2STGNN separates diffusion and inherent signals through gating and residual decomposition and models the two types of dynamics separately [14]. TESTAM uses temporal, static-graph, and dynamic-graph experts, with gated Top-1 routing to adapt to changing spatiotemporal dependencies [15].

These models organize heterogeneous dynamics with temporal branches, signal decomposition, or expert routing. Our design assigns each evidence source a correction role and gates its contribution to a shared base prediction using the current state.

Under deviations from routine patterns, the relevant distinction is the role of each evidence source. Directed time-lagged associations encode cross-region relations, whereas node self-evolution preserves local continuity; their different temporal meanings call for separate processing paths.

2.3 Dynamic Dependencies and Historical Message Selection

Traffic forecasting also needs to select cross-region information as relations change with time. GMAN uses spatial and temporal attention and connects historical representations to future steps with transform attention [20]. PDFormer combines spatial self-attention with short- and long-range masks and uses delay-aware transformations to address temporal misalignment [16]. TrGNN extracts transition patterns from vehicle trajectories and examines nonrecurrent flow under peak-hour and exceptional-event conditions [28]. These studies show that traffic relations depend on temporal shifts and propagation delays as well as spatial distance [16], [29].

Adaptive graph learning infers latent spatial relations from data. MTGNN and AGCRN construct adjacency relations from learnable node embeddings [21], [22]. These relations identify candidate interactions; a forecasting model must still determine whether a particular historical message is useful for the current context and forecast horizon.

Memory mechanisms provide another way to adapt spatial relations and retrieve historical patterns. MegaCRN generates graph structures from learnable meta-node memory [30]; RAST retrieves fine-grained patterns from an external memory bank and fuses them through cross-attention [31]. Cross-region forecasting therefore involves two linked decisions: which regions and lags form the candidate set, and which messages should be selected for the current target.

The usefulness of cross-region historical information depends on its context [32]. A neighboring region can carry different predictive value at different lags, temporal scales, and forecast steps. Our BCMS module addresses this selection problem by conditioning message scores on source and target backgrounds and on the forecast step.

3 Problem Definition

We partition the study area into N regular grid cells and treat each cell as a forecasting node. At time t, the regional flow matrix is

$$X_t = \left[x_t^{\text{in}},\, x_t^{\text{out}}\right] \in \mathbb{R}^{N\times 2}, \tag{1}$$

where $x_t^{\text{in}}, x_t^{\text{out}} \in \mathbb{R}^N$ are the inflow and outflow during the sampling interval. They count trips ending and starting in each region, respectively [33]. A single model forecasts both channels, ordered as inflow followed by outflow. Sequence tensors use time, node, and channel order, with the batch dimension omitted.

Given a historical observation sequence ending at time t with length T,

$$X_{t-T+1:t} = \left[X_{t-T+1}, \ldots, X_t\right] \in \mathbb{R}^{T\times N\times 2}, \tag{2}$$

the goal is to jointly forecast inflow and outflow over the next H time steps:

$$Y_{t+1:t+H} = \left[X_{t+1}, \ldots, X_{t+H}\right] \in \mathbb{R}^{H\times N\times 2}. \tag{3}$$

Here, T and H are the history length and number of forecast steps. With a 5-minute sampling interval, we set $T = H = 12$, using the past 60 minutes to forecast the next 60 minutes.

We formulate regional traffic flow forecasting as learning the spatiotemporal mapping

$$\widehat{Y}_{t+1:t+H} = f_\theta(X_{t-T+1:t}, C_t; \mathcal{S}, \mathcal{B}) \in \mathbb{R}^{H\times N\times 2}, \tag{4}$$

Here, θ denotes the learnable parameters. The temporal information $C_t \in \mathbb{R}^{T\times 2}$ contains time of day and day of week for the historical window and is shared across nodes. The fixed spatial supports $\mathcal{S}$ include self-relations, flow-specific directed time-lagged associations, and candidate cross-region neighborhoods. The lookup table $\mathcal{B}$ stores the routine-flow reference estimated from training data. Forecasts use observations up to t; $\widehat{Y}_{t+1:t+H}$ is expressed on the original flow scale.

For each historical time $u \in \{t-T+1, \ldots, t\}$, we retrieve the corresponding routine-flow reference from $\mathcal{B}$:

$$b_u = \left[b_u^{\text{in}},\, b_u^{\text{out}}\right] \in \mathbb{R}^{N\times 2}. \tag{5}$$

We estimate the reference from training observations grouped by node, channel, weekday or weekend, and time-of-day slot. The flow deviation is

$$D_u = X_u - b_u \in \mathbb{R}^{N\times 2}. \tag{6}$$

The deviation D_u measures the difference between observed flow and its routine level. Section 4 uses this reference with historical trajectories to represent trend deviation and short-term fluctuation.

The model corrects the base prediction when traffic deviates from routine patterns. It combines recent node dynamics with cross-region time-lagged associations, then selects historical messages using source and target backgrounds and the forecast horizon. Section 4 gives the residual refinement mechanism.

4 Methodology

4.1 Overall Framework

Figure 2 summarizes RBSRN. Shared dual-frequency encoding produces a base prediction. SERR then combines self-evolution and directed time-lagged experts, while BCMS selects cross-region historical messages from the source and target backgrounds for each forecast step. The two modules operate in parallel and add their corrections to the base prediction:

$$\widetilde{Y} = \widetilde{Y}^0 + \Delta\widetilde{Y}^S + \Delta\widetilde{Y}^B. \tag{7}$$

Here, $\widetilde{Y}^0$ is the base prediction, and $\Delta\widetilde{Y}^S$ and $\Delta\widetilde{Y}^B$ are the SERR and BCMS corrections, respectively. They correspond to Y_{Base}, ΔState, and ΔMessage in the figure. All three belong to $\mathbb{R}^{H\times N\times 2}$ and use the scale standardized separately for each flow channel, as indicated by the tilde. Below, we reindex historical positions as $k = 1, \ldots, T$, denote forecast steps by $h = 1, \ldots, H$, and distinguish flow channels using $c \in \{\text{in}, \text{out}\}$.

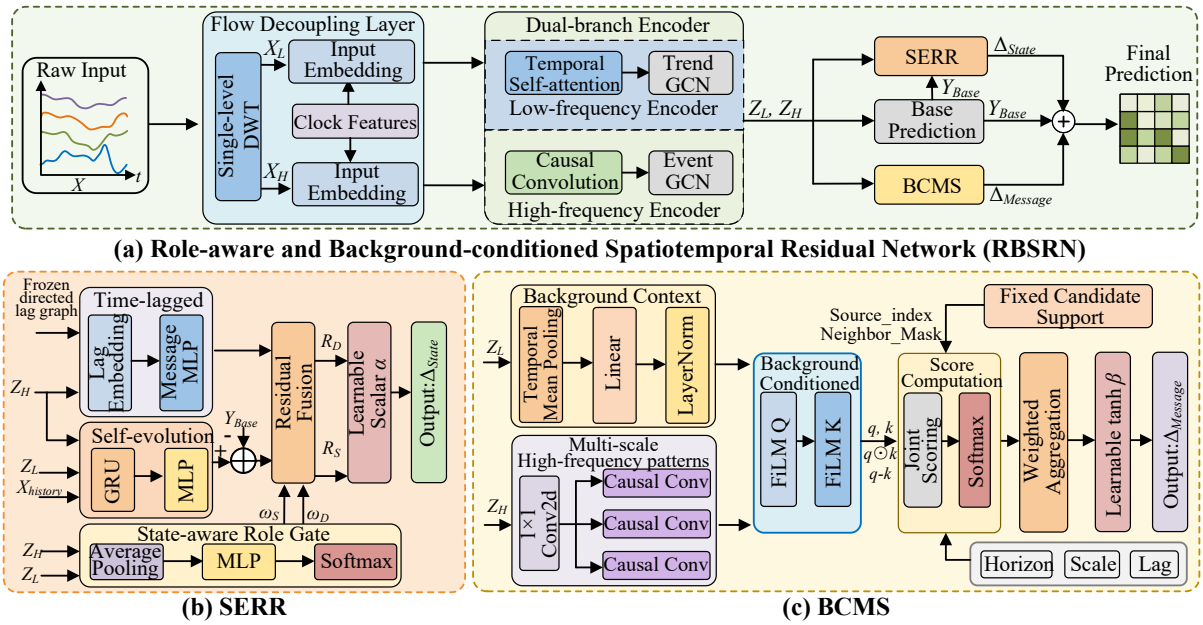


Figure 2 RBSRN architecture. (a) Base prediction and parallel residual fusion. (b) SERR. (c) BCMS.

4.2 Shared Dual-Frequency Spatiotemporal Representations

RBSRN separates smooth changes from short-term fluctuations with low- and high-frequency branches. We apply single-level discrete wavelet decomposition and reconstruction along the historical time dimension of each flow channel, standardize both sequences with the training statistics, and append time-of-day and day-of-week features to obtain $U_L, U_H \in \mathbb{R}^{T\times N\times 4}$. The branches share the input projection and temporal embeddings. The low-frequency branch uses temporal self-attention and trend graph encoding; the high-frequency branch uses temporal convolution and event graph encoding:

$$Z_L = \mathrm{F}_L(U_L), \tag{8}$$

$$Z_H = \mathrm{F}_H(U_H), \tag{9}$$

where $Z_L, Z_H \in \mathbb{R}^{T\times N\times d_b}$ and d_b denotes the shared representation dimension. We fuse the two feature streams using a gated linear unit (GLU) and a rectified linear unit (ReLU) to obtain the base prediction:

$$\widetilde{Y}^0 = \mathrm{R}(\mathrm{ReLU}[Z_L + \mathrm{GLU}(Z_L) + Z_H])\,. \tag{10}$$

The convolutional prediction head R spans the full historical window and maps the features to inflow and outflow predictions for the next H steps. SERR and BCMS use Z_L before the GLU operation, together with Z_H, as their shared inputs.

4.3 State-Evidence Expert Residual Refinement (SERR)

SERR separates node self-evolution from cross-region time-lagged associations and allocates their corrections according to the current state (Figure 2(b)). Let $x_{k,i,c}$ denote the raw flow at historical position k, node i, and channel c, and let $b_{k,i,c}$ be its routine-flow reference estimated

from training data. We define the state evidence as

$$\begin{aligned} \bar{x}_{k,i,c} &= \frac{1}{\ell_k} \sum_{r=k-\ell_k+1}^{k} x_{r,i,c}, \qquad \ell_k = \min(3,k), \\ e_{i,c} &= \left[\bar{x}_{T,i,c} - b_{T,i,c},\ x_{T,i,c} - \bar{x}_{T,i,c}\right]^{\mathsf{T}}. \end{aligned} \tag{11}$$

The two components of $e_{i,c} \in \mathbb{R}^2$ describe trend deviation and short-term fluctuation at the final historical time step, respectively. The mean uses only observed positions within the current historical window and adjusts its length to the available observations at the window's start.

The self-evolution expert encodes each node's recent trajectory. For each flow channel, it concatenates raw flow with the sum of the shared dual-frequency features, feeds the sequence to a shared gated recurrent unit (GRU), and maps the final hidden state to a multi-step prediction:

$$P^{\text{self}}_{:,i,c} = \mathcal{P}_S\big(\psi_S\big(\text{GRU}_{\text{last}}\left([Z_{L,k,i} + Z_{H,k,i} \,\|\, x_{k,i,c}]_{k=1}^{T}\right)\big)\big). \tag{12}$$

Here, $\|$ denotes feature concatenation, and ψ_S is a two-layer perceptron with tanh activation. The projection $\mathcal{P}_S$ linearly maps d_b-dimensional features to predictions over H steps. This branch further extracts each node's temporal evolution from the shared spatiotemporal representations.

The directed time-lagged expert retrieves source-node features through the fixed association set $\mathcal{E}_c$. For an edge $(j,i,\ell) \in \mathcal{E}_c$, j is the source, i the target, and ℓ a historical lag with $0 < \ell < T$. A lag embedding modulates the source high-frequency feature, which is combined with target state evidence to form a message:

$$\begin{aligned} \eta_{i,c} &= W_e e_{i,c}, \\ m_{j\to i,\ell,c} &= \psi_D([(1+\tanh v_\ell) \odot Z_{H,T-\ell,j} \,\|\, \eta_{i,c}]), \\ R^{\text{delay}}_{:,i,c} &= \mathcal{P}_D\left(\eta_{i,c} + \sum_{(j,i,\ell)\in\mathcal{E}_c} a^c_{ji\ell} m_{j\to i,\ell,c}\right). \end{aligned} \tag{13}$$

Here, $W_e \in \mathbb{R}^{d_b\times 2}$ projects the evidence, and $v_\ell \in \mathbb{R}^{d_b}$ is a learnable lag embedding. The message transformation ψ_D uses a sigmoid linear unit (SiLU) activation without bias, $a^c_{ji\ell}$ is a fixed edge weight, and $\mathcal{P}_D$ is a multi-step linear output head. Each expert shares its own parameters across nodes and flow channels, while directed time-lagged associations are specified separately for each flow channel.

The state-conditioned gate uses temporal means of the low- and high-frequency features and the state evidence for both channels to determine the expert contributions:

$$\Omega_i = \text{Softmax}_{\text{expert}}\big(\text{reshape}_{2\times 2}\ g\big([\bar{Z}_{L,i} \,\|\, \bar{Z}_{H,i} \,\|\, e_{i,\text{in}} \,\|\, e_{i,\text{out}}]\big)\big). \tag{14}$$

The temporal means $\bar{Z}_{L,i}, \bar{Z}_{H,i}$ are d_b-dimensional vectors averaged over the historical window, and g is a two-layer perceptron with SiLU activation. In $\Omega_i \in \mathbb{R}^{2\times 2}$, rows correspond to flow channels, and columns correspond to the self-evolution and directed time-lagged experts. Softmax normalizes along the expert dimension to produce $\omega^S_{i,c}$ and $\omega^D_{i,c}$. These weights vary by sample, node, and flow channel but are shared across all forecast steps. The resulting residual

correction is

$$\Delta\widetilde{Y}^{S}_{h,i,c} = \alpha\left[\omega^{S}_{i,c}\left(P^{\text{self}}_{h,i,c} - \widetilde{Y}^{0}_{h,i,c}\right) + \omega^{D}_{i,c} R^{\text{delay}}_{h,i,c}\right]. \tag{15}$$

Both P^{self} and R^{delay} use the same standardized flow scale as the base prediction, and α is a global learnable scalar. The self-evolution expert contributes a residual relative to the base prediction, whereas the directed time-lagged expert provides a correction from cross-region historical information. The state-conditioned gate combines the contributions of these two experts.

4.4 Background-conditioned Cross-region Message Selection (BCMS)

BCMS selects historical messages from fixed candidate neighborhoods using the current background and forecast horizon (Figure 2(c)). It maps the low-frequency temporal mean to a node background representation and extracts multiscale high-frequency patterns:

$$\xi_i = \text{LN}\big(\mathcal{P}_b(\bar{Z}_{L,i})\big), \qquad z^s = \mathcal{T}_s(\mathcal{P}_H(Z_H)). \tag{16}$$

Here, $\xi_i \in \mathbb{R}^d$ and $z^s \in \mathbb{R}^{T\times N\times d}$, with $d = 32$. The operator $\mathcal{P}_b$ is a linear projection, LN denotes layer normalization, and $\mathcal{P}_H$ is a 1×1 convolution shared across scales. Each $\mathcal{T}_s$ consists of a left-padded depthwise temporal convolution, a pointwise convolution, and a Gaussian error linear unit (GELU) activation. The three scales use a kernel size of 3 and dilation rates of 1, 2, and 4, yielding temporal spans $r_s \in \{3, 5, 9\}$.

For node i, let $\mathcal{N}_i$ denote its fixed candidate source set and consider historical lags $\tau \in \{0, 1, 2, 3\}$. The valid candidate set is $\mathcal{A}_i = \{(j, \tau, s) : j \in \mathcal{N}_i,\ r_s + \tau \le T\}$, with at most 8 sources retained per node. At each scale, the query represents the target's final historical pattern, and the key represents a source historical pattern. Their respective low-frequency backgrounds modulate the query and key:

$$\begin{aligned} q^s_i &= (1 + \gamma_q(\xi_i)) \odot Q(z^s_{T,i}) + \delta_q(\xi_i), \\ k^s_{j,\tau} &= (1 + \gamma_k(\xi_j)) \odot K(z^s_{T-\tau,j}) + \delta_k(\xi_j). \end{aligned} \tag{17}$$

The operators Q, K are learnable affine mappings; γ and δ produce d-dimensional scaling and shifting parameters, respectively [34]. The source and target low-frequency backgrounds therefore condition the matching of high-frequency patterns.

For forecast step h, we define the joint score as

$$\begin{aligned} u_{i,j,\tau,s} &= A_q q^s_i + A_k k^s_{j,\tau} + A_p(q^s_i \odot k^s_{j,\tau}) + A_d(q^s_i - k^s_{j,\tau}) + b_a, \\ a_{h,i,j,\tau,s} &= w_a^{\mathsf{T}}\,\text{GELU}(u_{i,j,\tau,s} + e_{\text{lag}}(\tau) + e_{\text{scale}}(r_s) + e_{\text{hor}}(h)). \end{aligned} \tag{18}$$

Here, $A_q, A_k, A_p, A_d \in \mathbb{R}^{d\times d}$ and $b_a, w_a \in \mathbb{R}^d$ are learnable parameters. Each temporal encoding applies a bias-free linear projection to $[v, \sin(2\pi v), \cos(2\pi v)]^{\mathsf{T}}$, where v is the corresponding lag, temporal span, or forecast horizon in hours. The forecast-step encoding lets candidate contributions vary with the horizon.

We add a zero-message option, with score and value set to zero, to control the total contribution of historical information. For each actual candidate, joint normalization gives the weight

$$w_{h,i,j,\tau,s} = \frac{\exp(a_{h,i,j,\tau,s})}{1 + \sum_{(j',\tau',s')\in\mathcal{A}_i} \exp(a_{h,i,j',\tau',s'})}. \tag{19}$$

The denominator's constant 1 is the zero-message option, whose probability mass reduces the total weight assigned to actual messages. We project source high-frequency patterns to values, aggregate them with the selection weights, and generate corrections for both channels:

$$\begin{aligned} m_{h,i} &= \sum_{(j,\tau,s)\in\mathcal{A}_i} w_{h,i,j,\tau,s} W_v z^s_{T-\tau,j}, \\ \Delta\widetilde{Y}^B_{h,i,:} &= \tanh(\beta)\, W_o m_{h,i}. \end{aligned} \tag{20}$$

Both $W_v \in \mathbb{R}^{d\times d}$ and $W_o \in \mathbb{R}^{2\times d}$ are bias-free projections, and β is a global learnable scalar. Inflow and outflow share the message selection weights and aggregated representation, while the two-output head produces their respective corrections. BCMS generates weights for each forecast step, complementing the SERR expert weights shared across forecast steps.

4.5 Joint Forecasting Output

We add both corrections to the base prediction using Eq. (7) and restore the original flow scale with fixed channel-wise statistics from training input windows:

$$\widehat{Y}_{t+h,i,c} = \sigma_c \left(\widetilde{Y}^0_{h,i,c} + \Delta\widetilde{Y}^S_{h,i,c} + \Delta\widetilde{Y}^B_{h,i,c} \right) + \mu_c. \tag{21}$$

Here, μ_c and σ_c are the mean and standard deviation of channel c. We initialize α and β to zero, so the initial output equals the base prediction. We also zero-initialize the final SERR gating layer, giving both experts an initial weight of 0.5. Training then learns the two correction pathways jointly for inflow and outflow.

5 Experiments and Analysis

5.1 Experimental Settings

5.1.1 Datasets, Preprocessing, and Forecasting Protocol

We evaluated regional traffic flow forecasting on two datasets: Chengdu DiDi ride-hailing orders collected in November 2016 and Shanghai taxi origin–destination (OD) records collected in August 2016. For each city, we selected a fixed core area and divided it into a 32×32 grid. Inflow and outflow are counts per 5 min, aggregated independently from trip destinations and origins, respectively. Table 1 summarizes the datasets, and Figures 3 and 4 show their endpoint distributions, grid partitions, and mean flows.

Both datasets used the same cleaning and aggregation procedure, with timestamps converted to Beijing time (UTC+8). We removed exact duplicates and records with invalid timestamps or coordinates. Retained orders lasted 2–120 minutes, had both endpoints within the

city's coarse geographic boundary, and had an average speed no greater than $120\,\mathrm{km/h}$, computed from great-circle distance and trip duration. Valid endpoints in the core area were aggregated into grid cells and time intervals; cells with no endpoint events were assigned zero. Shanghai source quality was flagged as invalid from 2016-08-11 to 2016-08-14. We retained these records in the flow series and in the training-input statistics, while applying the quality mask when computing source-quality summaries and scenario references.

Table 1 Dataset properties and sample splits

Property	Chengdu	Shanghai
Observation period	2016-11-01 to 2016-11-30	2016-08-01 to 2016-08-31
Grid size	32×32	32×32
Approx. cell dimensions (km)	0.42×0.44	0.80×0.68
Sampling interval	5 minutes	5 minutes
Complete time intervals	8,640	8,928
Orders after cleaning	6,082,753	7,744,423
Training windows	5,170	5,343
Validation windows	1,724	1,781
Test windows	1,723	1,781

For each dataset, we generated stride-1 rolling-origin windows. Each window uses the past 12 observations to predict inflow and outflow over the next 12 steps, covering 60 minutes of history and forecasts. We retained the associated time-of-day and day-of-week features and kept only windows with complete inputs and targets. The resulting windows were ordered by forecast origin and divided chronologically into training, validation, and test sets at an approximate ratio of 6:2:2 (Table 1). This rolling-origin protocol preserves temporal order while allowing each forecast origin to use the immediately preceding observations as history. All methods used the same ordered indices and forecasting task.

For each flow channel, standardization means and standard deviations were computed from the historical inputs of training windows and fixed for validation and testing. Predictions were returned to the original flow scale by channel-wise inverse standardization. Section 5.1.3 describes checkpoint selection, and Section 5.1.2 defines the evaluation metrics.

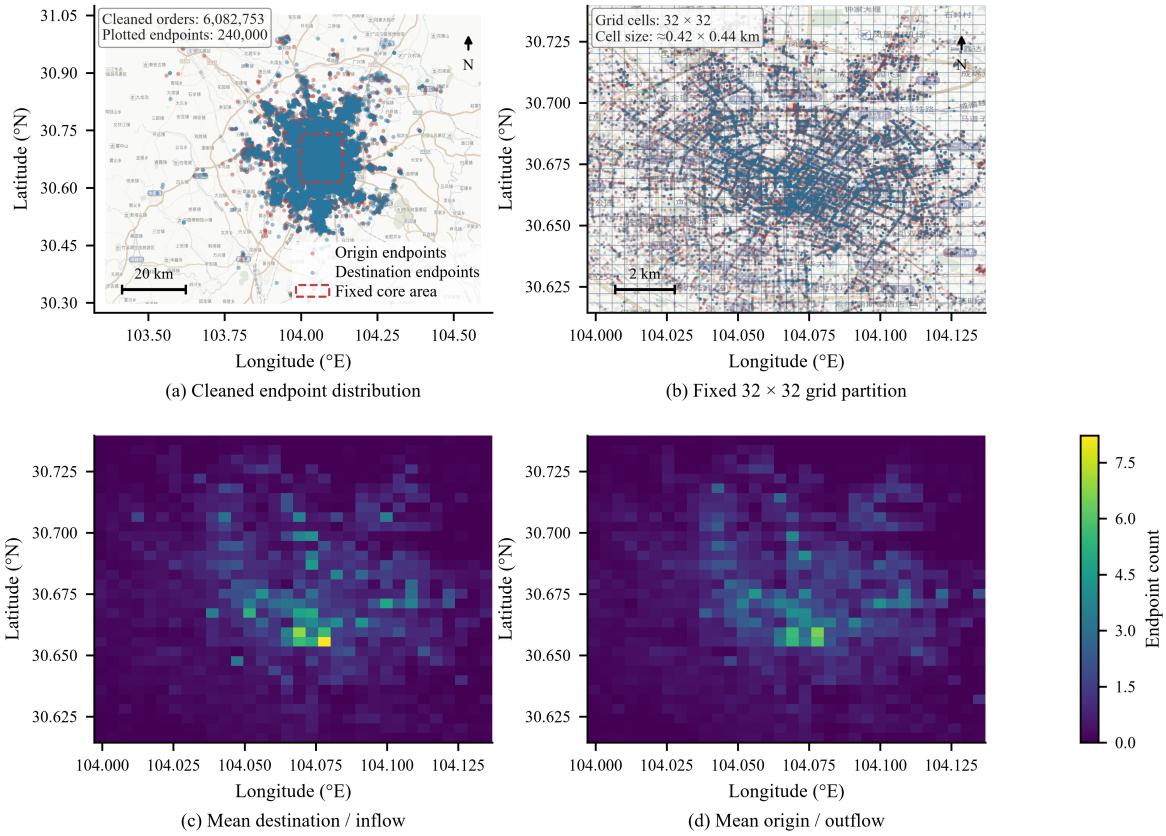


Figure 3 Chengdu endpoint distribution, grid partition, and mean flow.

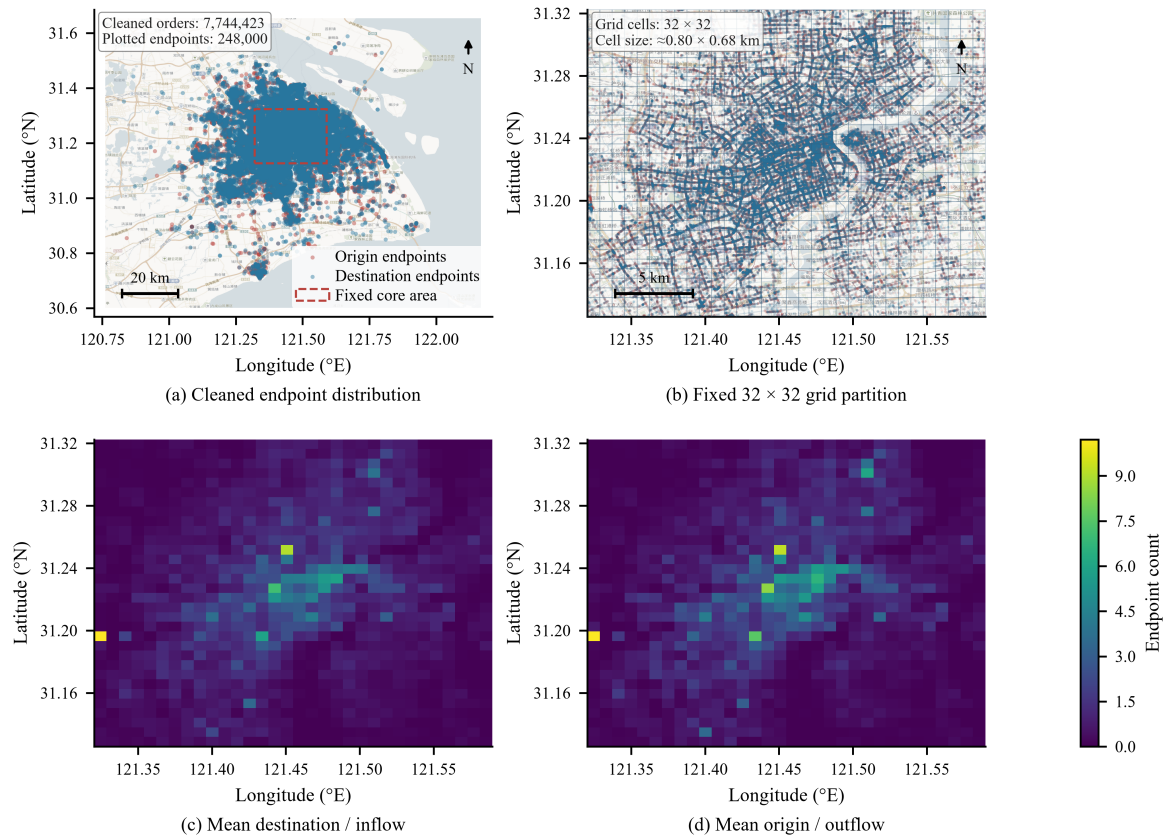


Figure 4 Shanghai endpoint distribution, grid partition, and mean flow.

5.1.2 Baseline Methods and Evaluation Metrics

The main experiments compared RBSRN with 13 baselines from three forecasting paradigms. Conventional methods were historical average (HA) and support vector regression (SVR). Deep models were DCRNN [7], STGCN [8], Graph WaveNet (GWNet) [9], DSTAGNN [35], PDFormer [16], STDN [36], MetaDG [37], HyperD [26], and RAST [31]; general time-series models were PatchTST [38] and TimeMixer [39]. Every method used the chronological split and 12-step task defined above. Inflow and outflow were evaluated separately in each city.

The scenario analysis additionally included vector autoregression (VAR), STNorm [40], and STWave [10], for 16 baselines in total.

We use mean absolute error (MAE) and root mean squared error (RMSE), computed on the original flow scale after inverse standardization. The main tables use a positive-target mask and evaluate locations with observed flow greater than zero. The scope-sensitivity analysis applies the same RBSRN checkpoint to all grid cells, including zero targets; it characterizes metric sensitivity rather than reranking baselines. Let $\mathcal{T}_{\text{test}}$ denote the forecast origins of the actual test windows. Let $y_{t,h,i,c}$ and $\widehat{y}_{t,h,i,c}$ denote the observed and predicted values for forecast origin t, forecast step h, grid cell i, and channel $c \in \{\text{in}, \text{out}\}$. The prediction error is $e_{t,h,i,c} = \widehat{y}_{t,h,i,c} - y_{t,h,i,c}$. The two evaluation scopes are

$$\Omega^{+}_{h,c} = \big\{(t,i) \mid t \in \mathcal{T}_{\text{test}},\ 1 \le i \le N,\ y_{t,h,i,c} > 0\big\},$$
$$\Omega^{\text{all}}_{h,c} = \mathcal{T}_{\text{test}} \times \{1, \dots, N\}.$$

For $m \in \{+, \text{all}\}$, we pool all test windows and nodes within the corresponding scope to compute

$$\text{MAE}^{m}_{h,c} = \frac{1}{|\Omega^{m}_{h,c}|} \sum_{(t,i)\in\Omega^{m}_{h,c}} |e_{t,h,i,c}|, \tag{22}$$

$$\text{RMSE}^{m}_{h,c} = \sqrt{\frac{1}{|\Omega^{m}_{h,c}|} \sum_{(t,i)\in\Omega^{m}_{h,c}} e^{2}_{t,h,i,c}}. \tag{23}$$

Here, $|\Omega^{m}_{h,c}|$ is the number of evaluated locations. Both scopes exclude duplicate windows introduced by test-batch padding, and inflow and outflow are evaluated separately.

We report the single-step results H3, H6, and H12, corresponding to forecast steps 3, 6, and 12, or forecast horizons of 15, 30, and 60 minutes. AVG12 denotes the arithmetic mean of the metrics over all 12 forecast steps:

$$\text{AVG12-}\mathcal{M}^{m}_{c} = \frac{1}{12} \sum_{h=1}^{12} \mathcal{M}^{m}_{h,c}, \qquad \mathcal{M} \in \{\text{MAE}, \text{RMSE}\}. \tag{24}$$

For AVG12 RMSE, we compute RMSE at each forecast step and then average those 12 values.

5.1.3 Implementation and Computing Environment

Model experiments used a single NVIDIA A100-SXM4-40GB GPU. Table 2 lists the hardware, software, and training settings. All configurations used the same data split, a maximum of 200 epochs, and an early-stopping patience of 20 epochs. We selected the checkpoint with the lowest joint validation MAE and reported MAE and RMSE at that checkpoint.

Table 2 Computing environment and training settings

Item	Setting
Optimizer and learning rate	Ranger; fixed at 0.001, with no learning-rate schedule
Weight decay	10^{-4}; no decay for the BCMS parameter β
Batch size	64 for both training and validation
Training budget and checkpoint selection	At most 200 epochs, with an early-stopping patience of 20 epochs; checkpoint selected by the lowest joint validation MAE
GPU	One NVIDIA A100-SXM4-40GB, with 40 GB of GPU memory
CPU	Two Intel Xeon Gold 6138 processors @ 2.00 GHz, with 40 cores and 80 threads in total
Software environment	Python 3.11.15; PyTorch 2.7.1+cu128; CUDA 12.8 (PyTorch build version)

5.2 Main Experimental Results

This section evaluates overall accuracy, spatial prediction, performance across deviation levels, and the effects of the two residual pathways. The first two analyses use the full test set; the latter analyses use fixed-model diagnostics to relate correction benefits to traffic conditions.

5.2.1 Overall Prediction Performance

Tables 3–6 compare RBSRN with 13 baselines on Chengdu and Shanghai inflow and outflow under positive-target evaluation. Relative changes use unrounded values and the strongest baseline for each task and metric. Bold and underlined values mark the best and second-best results in each column.

RBSRN achieved the lowest AVG12 MAE and RMSE on all four tasks. For Chengdu inflow, Chengdu outflow, Shanghai inflow, and Shanghai outflow, AVG12 MAE decreased by 2.00%, 0.46%, 1.18%, and 0.43%, respectively, relative to the strongest baseline. The corresponding RMSE reductions were 5.01%, 1.50%, 2.33%, and 0.06%; the strongest baselines were STGCN, STGCN, HyperD, and HyperD.

Across horizons, RBSRN achieved the lowest MAE and RMSE at H3, H6, and H12 for inflow in both cities. For outflow, HyperD achieved the best H12 MAE in both cities and the best H12 RMSE in Shanghai; RBSRN was best in the remaining reported comparisons.

Table 3 Inflow forecasting results for Chengdu (32×32 grid)

	H3		H6		H12		AVG12	
Model	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE
HA	1.053	1.522	1.083	1.600	1.144	1.739	1.088	1.609
SVR	0.810	1.354	0.835	1.391	0.892	1.499	0.840	1.405
DCRNN (ICLR'18)	0.725	1.321	0.790	1.363	1.025	1.574	0.829	1.399
STGCN (IJCAI'18)	0.721	1.342	0.724	1.348	0.743	1.409	0.728	1.362
GWNet (IJCAI'19)	0.719	1.332	0.731	1.373	0.756	1.450	0.733	1.378
DSTAGNN (ICML'22)	1.022	1.461	1.008	1.458	1.044	1.505	1.027	1.473
PDFormer (AAAI'23)	1.002	1.443	1.013	1.473	1.030	1.515	1.014	1.473
PatchTST (ICLR'23)	0.799	1.405	0.826	1.483	0.880	1.614	0.831	1.490
TimeMixer (ICLR'24)	0.818	1.434	0.844	1.505	0.895	1.620	0.848	1.510
STDN (AAAI'25)	1.178	1.589	1.180	1.593	1.179	1.600	1.178	1.593
MetaDG (AAAI'26)	1.102	1.564	1.170	1.678	1.291	1.882	1.178	1.692
HyperD (AAAI'26)	0.723	1.369	0.723	1.369	0.726	1.382	0.723	1.369
RAST (AAAI'26)	0.724	1.332	0.734	1.372	0.756	1.446	0.736	1.377
RBSRN (Ours)	**0.704**	**1.282**	**0.707**	**1.290**	**0.717**	**1.318**	**0.708**	**1.294**

Table 4 Outflow forecasting results for Chengdu (32×32 grid)

	H3		H6		H12		AVG12	
Model	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE
HA	1.023	1.404	1.043	1.439	1.085	1.510	1.047	1.445
SVR	0.792	1.239	0.809	1.251	0.838	1.285	0.810	1.255
DCRNN (ICLR'18)	0.714	1.230	0.761	1.237	1.151	1.642	0.844	1.326
STGCN (IJCAI'18)	0.714	1.235	0.716	1.239	0.725	1.258	0.718	1.242
GWNet (IJCAI'19)	0.713	1.253	0.718	1.265	0.729	1.289	0.719	1.267
DSTAGNN (ICML'22)	1.018	1.383	1.007	1.376	1.035	1.410	1.021	1.390
PDFormer (AAAI'23)	0.987	1.365	0.990	1.374	0.997	1.391	0.990	1.374
PatchTST (ICLR'23)	0.786	1.300	0.802	1.333	0.837	1.396	0.805	1.337
TimeMixer (ICLR'24)	0.807	1.315	0.823	1.345	0.856	1.404	0.826	1.349
STDN (AAAI'25)	1.135	1.501	1.135	1.501	1.135	1.503	1.134	1.500
MetaDG (AAAI'26)	1.082	1.471	1.139	1.543	1.245	1.672	1.146	1.550
HyperD (AAAI'26)	0.712	1.249	0.712	1.249	**0.714**	1.254	0.712	1.249
RAST (AAAI'26)	0.718	1.242	0.724	1.257	0.735	1.282	0.724	1.258
RBSRN (Ours)	**0.706**	**1.220**	**0.708**	**1.222**	0.715	**1.234**	**0.709**	**1.223**

The proportion of zero targets is calculated over the same test evaluation positions. For channel c, it is defined as

$$\rho_c = 1 - \frac{\sum_{h=1}^{12} |\Omega_{h,c}^{+}|}{12N|\mathcal{T}_{\text{test}}|}. \tag{25}$$

The denominator counts the evaluation positions across all actual test windows, 12 forecast steps, and 1,024 grid cells. When the same timestamp appears in multiple forecast windows, each corresponding forecasting task is counted separately. Table 7 reports the zero-target proportions and results under both evaluation scopes for each city and channel.

Table 7 compares AVG12 results under positive-target and all-grid evaluation using the same RBSRN checkpoint as the main tables. Zero targets accounted for 64.17%–71.68% of evaluation positions. Including them increased MAE and decreased RMSE in every task, so

Table 5 Inflow forecasting results for Shanghai (32×32 grid)

	H3		H6		H12		AVG12	
Model	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE
HA	1.019	1.437	1.040	1.490	1.080	1.593	1.043	1.498
SVR	0.817	1.251	0.837	1.274	0.879	1.344	0.841	1.283
DCRNN (ICLR'18)	0.734	1.243	0.748	1.262	0.808	1.328	0.758	1.272
STGCN (IJCAI'18)	0.741	1.257	0.743	1.260	0.754	1.293	0.745	1.268
GWNet (IJCAI'19)	0.735	1.268	0.742	1.288	0.760	1.343	0.744	1.295
DSTAGNN (ICML'22)	1.010	1.410	0.950	1.360	1.009	1.426	0.994	1.402
PDFormer (AAAI'23)	0.970	1.356	0.977	1.369	0.995	1.413	0.979	1.376
PatchTST (ICLR'23)	0.800	1.333	0.816	1.382	0.849	1.480	0.819	1.392
TimeMixer (ICLR'24)	0.821	1.331	0.836	1.372	0.868	1.463	0.839	1.381
STDN (AAAI'25)	1.219	1.651	1.220	1.659	1.219	1.683	1.218	1.661
MetaDG (AAAI'26)	1.082	1.517	1.168	1.624	1.421	1.944	1.201	1.667
HyperD (AAAI'26)	0.737	1.261	0.737	1.261	0.737	1.263	0.737	1.260
RAST (AAAI'26)	0.736	1.245	0.740	1.259	0.749	1.289	0.741	1.261
RBSRN (Ours)	**0.726**	**1.225**	**0.727**	**1.229**	**0.733**	**1.243**	**0.728**	**1.231**

Table 6 Outflow forecasting results for Shanghai (32×32 grid)

	H3		H6		H12		AVG12	
Model	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE
HA	1.059	1.528	1.079	1.559	1.118	1.629	1.082	1.565
SVR	0.866	1.373	0.882	1.395	0.914	1.447	0.883	1.398
DCRNN (ICLR'18)	0.782	1.355	0.794	1.365	0.856	1.388	0.804	1.365
STGCN (IJCAI'18)	0.784	1.353	0.787	1.359	0.797	1.385	0.789	1.364
GWNet (IJCAI'19)	0.783	1.381	0.788	1.394	0.801	1.424	0.789	1.396
DSTAGNN (ICML'22)	1.017	1.462	0.994	1.446	1.080	1.535	1.016	1.469
PDFormer (AAAI'23)	1.010	1.459	1.017	1.472	1.034	1.511	1.019	1.478
PatchTST (ICLR'23)	0.848	1.439	0.863	1.465	0.897	1.533	0.866	1.473
TimeMixer (ICLR'24)	0.869	1.448	0.883	1.473	0.915	1.538	0.886	1.480
STDN (AAAI'25)	1.109	1.591	1.107	1.593	1.105	1.605	1.106	1.595
MetaDG (AAAI'26)	1.127	1.600	1.245	1.748	1.599	2.238	1.298	1.825
HyperD (AAAI'26)	0.778	1.344	0.778	1.344	**0.778**	**1.347**	0.778	1.344
RAST (AAAI'26)	0.787	1.366	0.792	1.380	0.804	1.409	0.793	1.381
RBSRN (Ours)	**0.772**	**1.339**	**0.774**	**1.342**	0.779	1.353	**0.775**	**1.343**

Table 7 RBSRN sensitivity to evaluation scope (AVG12)

City	Channel	Zero targets (%)	Positive targets ($y > 0$)		All grid cells (including zero targets)	
			MAE	RMSE	MAE	RMSE
Chengdu	Inflow	71.68	0.708421	1.293545	0.936965	1.117916
	Outflow	70.13	0.708654	1.223468	0.936962	1.105671
Shanghai	Inflow	64.17	0.728085	1.230839	0.937587	1.139290
	Outflow	64.94	0.774534	1.342986	0.954306	1.184390

metric values depend on the evaluation scope. Baseline rankings in the main tables refer to positive-target evaluation; the all-grid results are reported for RBSRN only.

5.2.2 Spatial Prediction Performance

To assess spatial localization, we evaluated predictions 30 minutes after the forecast origin (H6). The analysis covered 1,723 Chengdu windows and 1,781 Shanghai windows. For each window, we computed MAE, Spearman rank correlation, and hotspot intersection over union (IoU) over all 1,024 grid cells, then averaged each metric over defined windows. Inflow, outflow, and their cellwise sum were evaluated separately. MAE used inverse-standardized predictions in counts per 5 min; rank correlation used average ranks for ties, and constant-observation windows contributed only to MAE and IoU.

For each map, hotspots were positive-flow cells within the top q fraction of the flow ranking. The threshold was the $\lceil qN \rceil$-th largest flow value, and positive values tied at the threshold were included. IoU was the intersection divided by the union and was undefined for an empty union.

Table 8 Spatial prediction statistics at H6 on the full test set

City	Flow	Windows	All-grid MAE	Spearman	Valid correlations	$\text{IoU}_{10\%}$	$\text{IoU}_{5\%}$
Chengdu	Inflow	1,723	0.9366	0.4327	1,723	0.3420	0.3495
	Outflow	1,723	0.9367	0.4503	1,723	0.3356	0.3147
	Total	1,723	1.6750	0.5324	1,723	0.4155	0.4142
Shanghai	Inflow	1,781	0.9382	0.4524	1,760	0.3224	0.3106
	Outflow	1,781	0.9549	0.4920	1,759	0.3547	0.3502
	Total	1,781	1.6722	0.5543	1,767	0.4068	0.4117

Table 8 reports mean total-flow rank correlations of 0.5324 in Chengdu and 0.5543 in Shanghai. At the top-10% threshold, the mean hotspot IoU was 0.4155 and 0.4068, respectively; separate inflow and outflow IoU values ranged from 0.3224 to 0.3547. At the top-5% threshold, mean total-flow IoU remained close to 0.41 in both cities. Window-to-window variation at H6 was larger than the variation in means across H1–H12: the H6 interquartile ranges were 0.270–0.537 in Chengdu and 0.331–0.519 in Shanghai, compared with mean ranges of 0.4122–0.4174 and 0.4048–0.4087.

Figure 5 shows total flow at 08:00 on each city's first complete H6 target day. In the Shanghai example, hotspot IoU was 0.4718. The predicted maximum was 23.02 counts per 5 min versus 18 observed counts per 5 min, indicating an overestimated peak while the predicted and observed hotspots still partially overlapped.

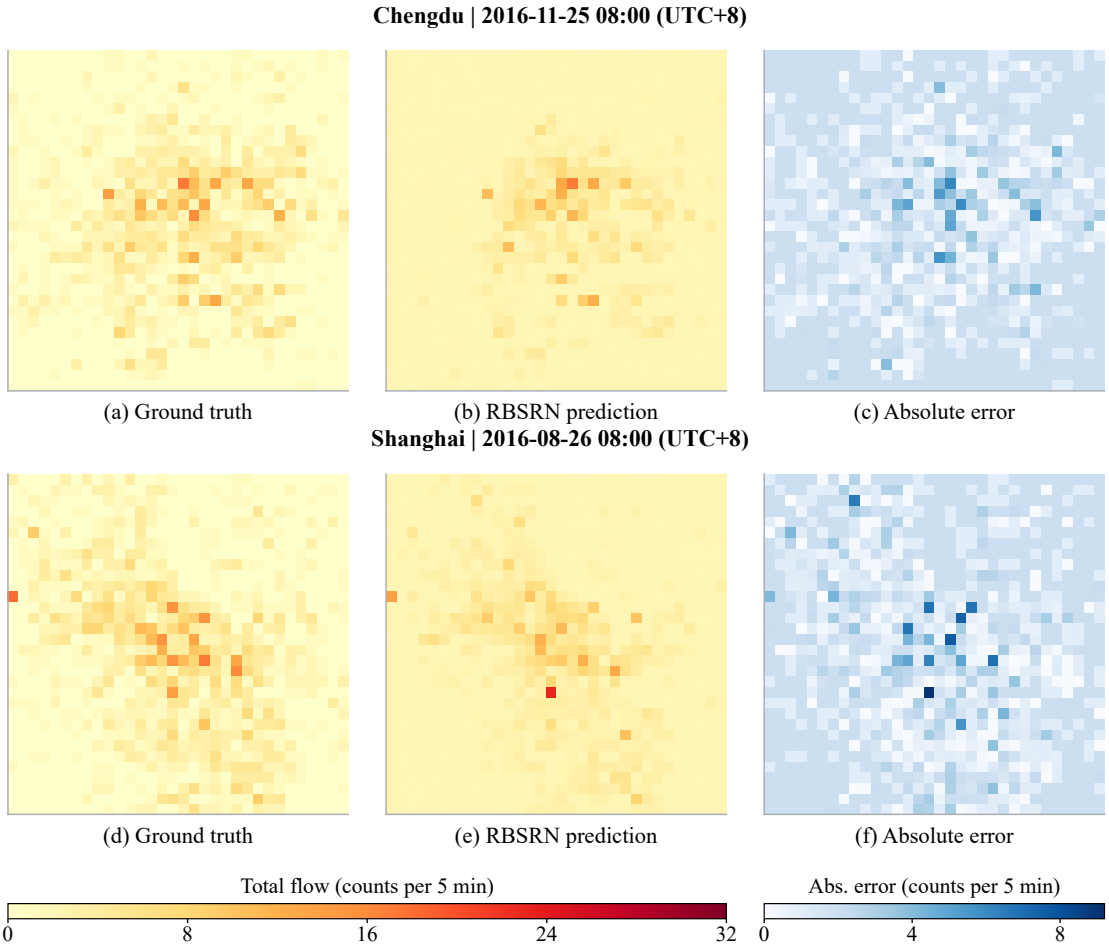


Figure 5 Total-flow predictions at H6 (08:00). Rows: Chengdu and Shanghai. Columns: observation, prediction, and absolute error.

5.2.3 Prediction Performance across Levels of Deviation from Routine Patterns

All-grid spatial statistics describe flow distribution at a given time. To examine prediction error under deviations from routine levels, we grouped observations by window, grid cell, and channel. At positions with positive H6 target flow, RBSRN was compared with 16 baselines. Chengdu scenario analysis and the subsequent residual diagnostics use a dedicated diagnostic checkpoint, whose full-test-set H6 MAE is 0.709952 for inflow and 0.709944 for outflow. The main Chengdu tables use the comparison checkpoint. Shanghai scenario analysis and residual diagnostics use the epoch-115 checkpoint reported in the main tables.

The routine-flow reference is the median training flow for the same grid cell, channel, 5-minute slot, and weekday/weekend type. We used deduplicated training-history timestamps that passed the quality checks. If a slot had fewer than four observations for one day type, the two types were pooled; if it had none, we used the grid-cell and channel median. We divided each observation's difference from its reference by a robust scale to obtain the standardized residual r_k. The scale is the training residual median absolute deviation multiplied by 1.4826, with a lower bound of 1 count. The input-window deviation score is $D = \frac{1}{12}\sum_{k=1}^{12}|r_k|$. For each city and channel, we pooled grid cells from training windows whose 12 historical steps passed the

quality checks. The 50th, 80th, and 95th percentiles of D define four groups (Table 9); boundary values enter the lower group.

Table 9 Scenario-group thresholds from training data

City	Channel	D_{50}	D_{80}	D_{95}	U_{75}	$B_{1/3}$	$B_{2/3}$
Chengdu	Inflow	0.166667	0.583333	1.083333	1.636364	0.279326	0.453125
Chengdu	Outflow	0.166667	0.625000	1.083333	1.636364	0.287192	0.471179
Shanghai	Inflow	0.281038	0.708333	1.096048	1.636364	0.311065	0.489387
Shanghai	Outflow	0.250000	0.674491	1.083333	1.590909	0.311294	0.492517

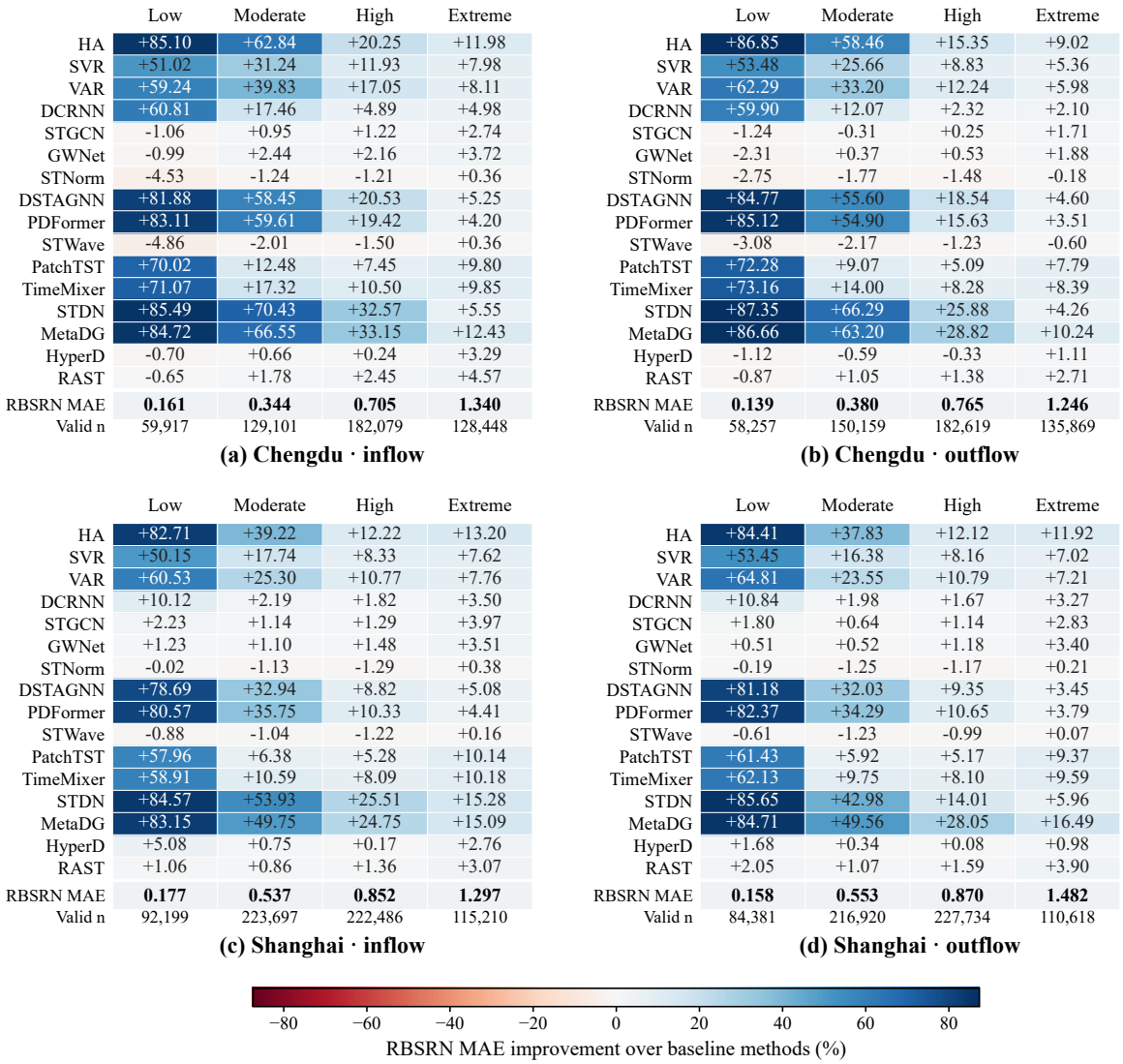


Figure 6 RBSRN relative MAE at H6 across deviation groups.

Figure 6 compares methods within each group; positive values indicate an MAE reduction relative to the corresponding baseline. In the Extreme group, RBSRN had the lowest MAE for Chengdu inflow, Shanghai inflow, and Shanghai outflow, with reductions of 0.36%, 0.16%, and 0.07% relative to STWave. For Chengdu outflow, RBSRN ranked third, with MAE 0.60% above STWave. STWave and STNorm had lower errors than RBSRN in the Moderate and High groups on all four tasks.

Absolute MAE increased with deviation level on all four tasks. Extreme-group MAE was 7.33–9.37 times the Low-group value. For Chengdu inflow, MAE increased from 0.161 to 1.340 while mean positive-target flow increased from 1.160 to 3.637 counts per 5 min, so the group differences reflect both deviation level and flow magnitude.

5.2.4 Effects of SERR Residual Corrections across Recent-Change Scenarios

Within the high-deviation subset combining High and Extreme observations ($D > D_{80}$), recent-change intensity is defined by successive differences in historical standardized residuals:

$$U = \frac{1}{11} \sum_{k=2}^{12} |r_k - r_{k-1}|. \tag{26}$$

The 75th percentile of U among high-deviation training samples separates Slow and Rapid groups; values above the threshold enter Rapid.

We examined each residual correction with a fixed-model residual-removal diagnostic. Model parameters, gates, and all other outputs were held fixed while one specified correction was subtracted from the final prediction:

$$\Delta\text{MAE}_e = \text{MAE}(\widehat{Y} - C_e) - \text{MAE}(\widehat{Y}), \tag{27}$$

Here, C_e is the signed correction added to the prediction, including the original gating weight, learnable strength coefficient, and inverse-standardization scale. Removing the SERR self-evolution expert subtracts its weighted difference from the base prediction; removing the directed time-lagged expert subtracts its weighted correction; removing BCMS subtracts the full cross-region correction. Gating weights retain their original values and are not renormalized. A positive difference means that the correction reduced group-average MAE. Because the loss is nonlinear, differences for separate corrections are not additive. This diagnostic complements the independently retrained ablations in Section 5.3: it measures sensitivity of one fixed model, whereas the ablations compare retrained configurations.

In Figure 7, all 16 removal differences across the eight city–channel–change groups were positive. The self-evolution correction benefit in Rapid was 1.15–2.82 times the Slow-group benefit, whereas the directed time-lagged correction was smaller. Its Rapid-group benefit was higher than its Slow-group benefit in Chengdu and lower in Shanghai.

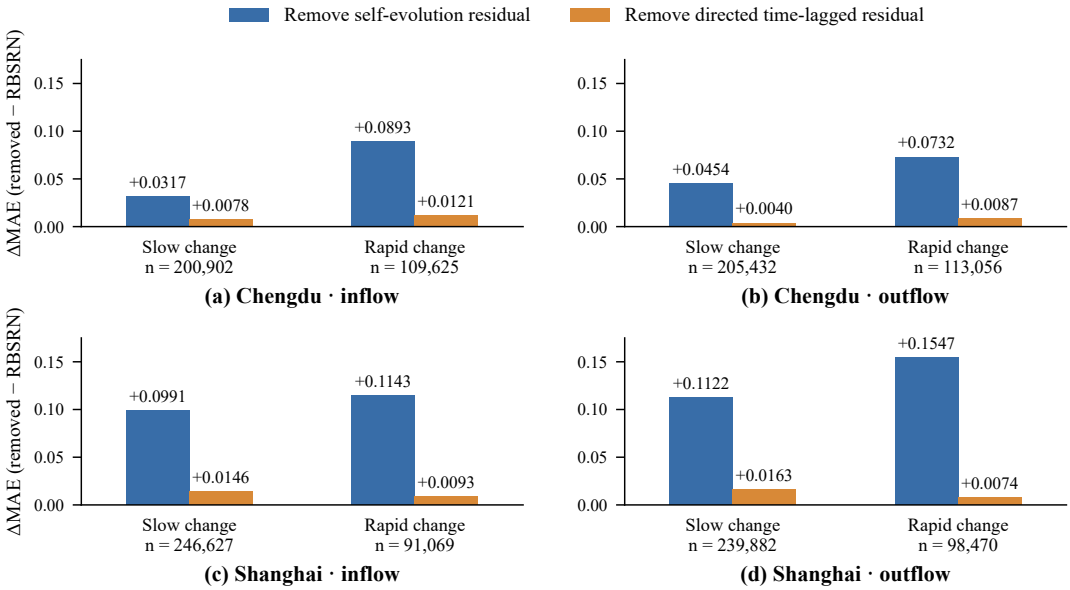


Figure 7 H6 SERR residual-removal sensitivity by recent-change group.

5.2.5 Effects of BCMS Residual Corrections across Background Discrepancies and Forecast Horizons

We measured source–target background similarity with a history-derived discrepancy proxy. Let $\mathcal{C}_i$ denote the valid fixed candidate-source set for target cell i, and let $\bar{r}_i$ be its mean standardized residual over the 12 historical steps. For each channel, define

$$B_i = \frac{1}{|\mathcal{C}_i|} \sum_{j \in \mathcal{C}_i} |\bar{r}_i - \bar{r}_j|. \tag{28}$$

The candidate set follows the model's fixed support. B_i describes discrepancy in the observed history and is separate from the low-frequency representation learned by BCMS. Tertile thresholds estimated from high-deviation training samples define Low, Medium, and High discrepancy groups (Table 9). We removed the full BCMS correction at H1–H12 using Eq. (27). Both figures report MAE changes in 10^{-3} counts per 5 min; each cell is $1000 \times \Delta$MAE.

Figure 8 shows a consistent ordering across city, channel, and horizon: the mean benefit was largest in the Low group, followed by Medium and High. Of 144 background-group–horizon cells, 143 were positive; the only negative value was approximately -4.8×10^{-6} for Chengdu outflow in the High group at H5. Benefits varied across horizons without a monotonic trend.

Figure 9 extends the analysis to the full test set with the same thresholds fitted from high-deviation training histories. All 144 removal differences were positive and exceeded the corresponding high-deviation values. The Low–Medium–High ordering persisted. For Shanghai inflow at H6, full-test differences were 0.025547, 0.015634, and 0.009873, compared with 0.013620, 0.009982, and 0.007434 in the high-deviation subset. The full-test analysis therefore shows that the average BCMS benefit extends beyond high-deviation samples.

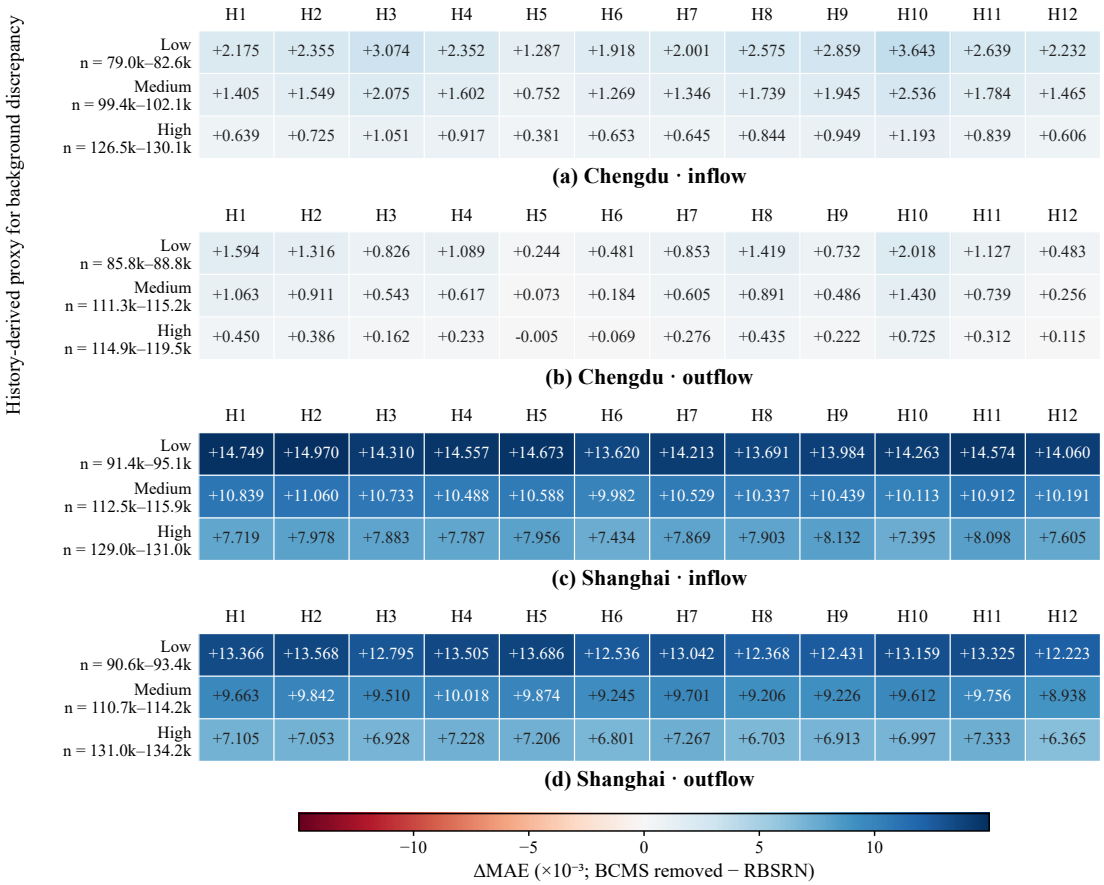


Figure 8 BCMS residual-removal sensitivity in high-deviation scenarios.

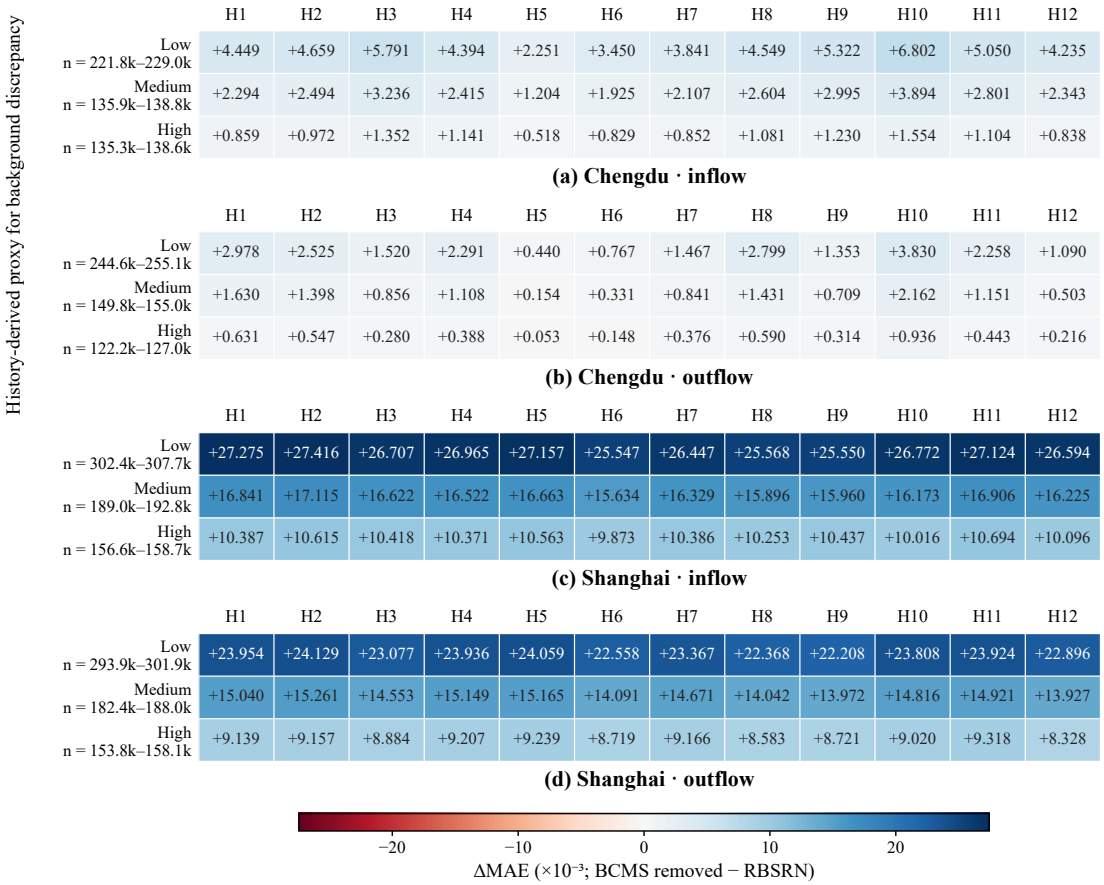


Figure 9 BCMS residual-removal sensitivity across the full test set.

5.3 Ablation Study

We evaluated the three residual pathways with Chengdu ablations. A1 removes the self-evolution expert, A2 removes the directed time-lagged expert, and A3 removes BCMS. Each variant was retrained for up to 200 epochs with an early-stopping patience of 20 epochs. In A1 and A2, the remaining expert weights were renormalized after setting the removed weight to zero. Testing used the positive-target mask and AVG12 aggregation defined in Section 5.1.2. Joint MAE and RMSE are the equal-weight averages of the inflow and outflow AVG12 values.

Table 10 Chengdu ablation results (AVG12)

Model configuration	Inflow MAE	Inflow RMSE	Outflow MAE	Outflow RMSE	Joint MAE	Joint RMSE
RBSRN	**0.7084**	**1.2935**	**0.7087**	**1.2235**	**0.7085**	**1.2585**
w/o Self-evolution expert (A1)	0.7129	1.3037	0.7121	1.2270	0.7125	1.2653
w/o Directed time-lagged expert (A2)	0.7105	1.3048	0.7099	1.2264	0.7102	1.2656
w/o BCMS (A3)	0.7109	1.3014	0.7102	1.2276	0.7105	1.2645

Table 10 shows that the full RBSRN had the lowest error on all six metrics. Removing any branch increased MAE and RMSE for both flows. Removing the self-evolution expert increased joint MAE by 0.56%, the largest increase among the variants. Removing the directed time-lagged expert increased joint RMSE by 0.56%, compared with 0.54% for removing self-evolution. The two experts therefore contribute through different error measures, with self-evolution having the larger effect on joint MAE.

Removing BCMS increased joint MAE and RMSE by 0.28% and 0.48%, respectively, with higher errors for both flows. The full model performed best when self-evolution, directed time-lagged association, and background-conditioned message selection were combined.

5.4 Hyperparameter Sensitivity Analysis

We conducted one-factor-at-a-time validation experiments on Chengdu to examine SERR gating capacity, BCMS representation capacity, and the initial values of the two residual pathways. The SERR gating and BCMS dimensions took values in $\{8, 16, 32, 64, 128\}$; α_0 and β_0 took values in $\{0, 0.001, 0.01, 0.05, 0.1\}$. These values initialize the learnable residual parameters in Sections 4.3 and 4.4, which continue to update during training. The reference configuration was $(32, 32, 0, 0)$, used in the main experiments. We varied one parameter at a time and kept the others fixed. Bold tick labels mark the reference configuration; minima were selected from unrounded results. Training and checkpoint selection followed Sections 5.1.3 and 5.1.1.

We built the reference model first and replaced the gating or BCMS module only when its dimension differed from 32. Residual initial values were assigned directly to the learnable parameters. We did not introduce additional microbatch splitting during training or validation.

The four parameters produced only small changes in joint validation error over the tested ranges (Figure 10). For each group, the relative range was the maximum–minimum difference divided by the reference error; it remained below 0.15% for joint MAE and 0.35% for joint RMSE.

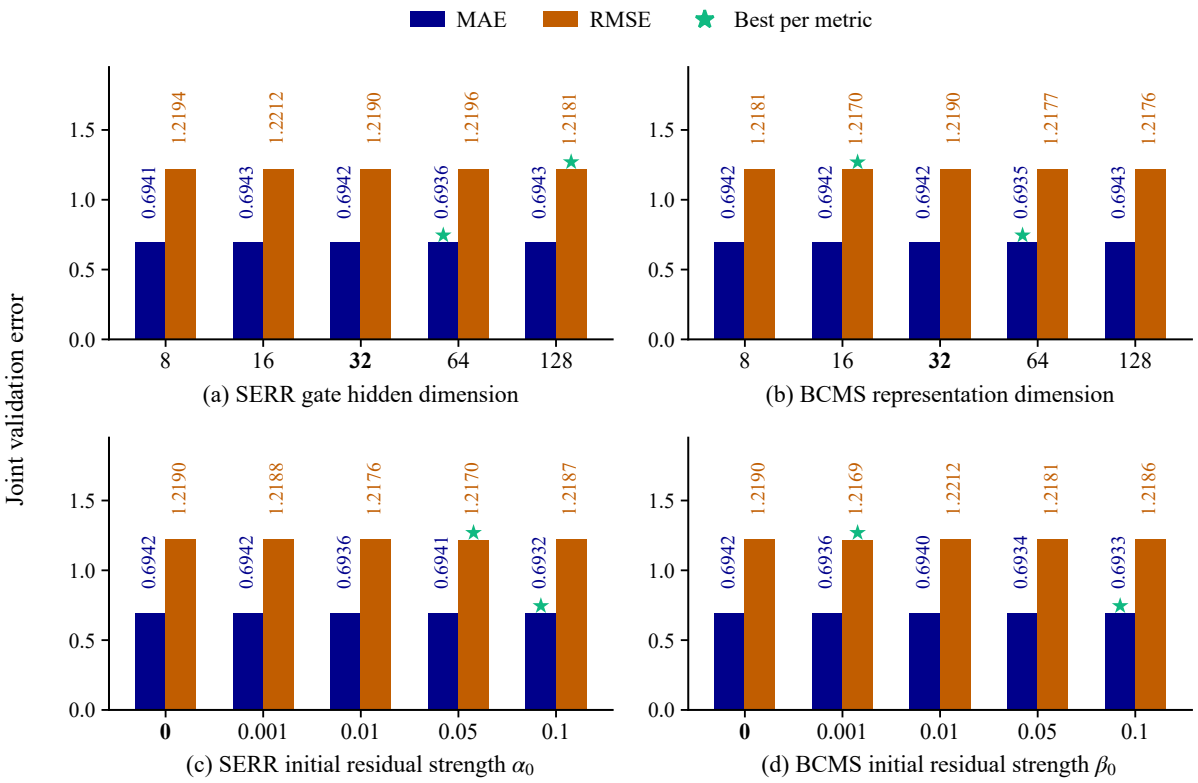


Figure 10 Joint validation error under hyperparameter changes on Chengdu.

Increasing the hidden dimension did not consistently reduce error, and the minima for joint MAE and RMSE occurred at different dimensions. Positive residual initializations improved over zero initialization only slightly, and the two metrics favored different initial values.

Figure 11 shows the channel-wise results. The flow channels responded differently to SERR capacity: inflow MAE was lowest at dimension 64, whereas outflow MAE was lowest at dimension 8.

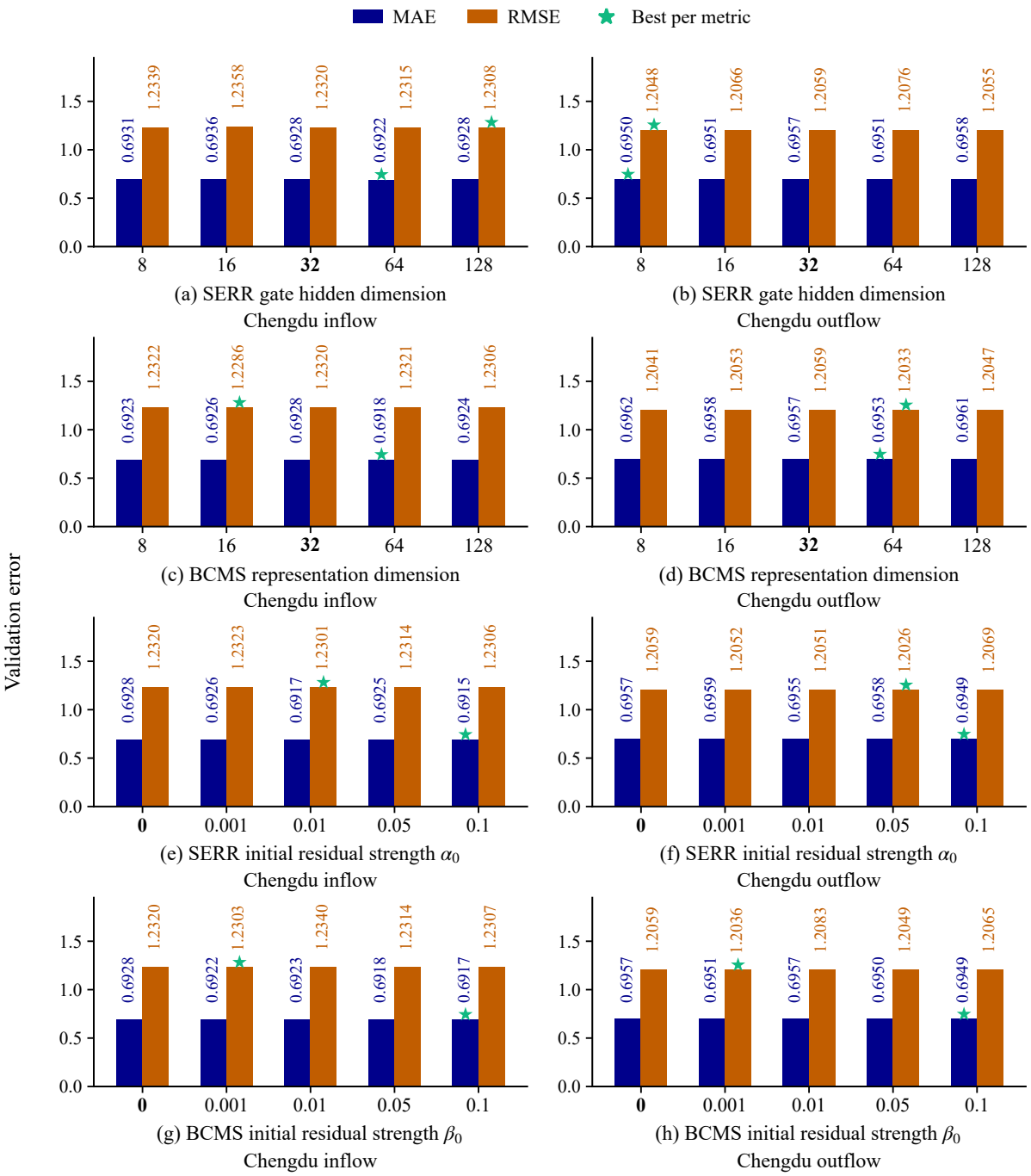


Figure 11 Channel-wise validation error under hyperparameter changes on Chengdu.

5.5 Model Size and Inference Efficiency

With the gating and BCMS dimensions set to 32, RBSRN has approximately 7.071 million trainable parameters. We measured inference efficiency with the Chengdu checkpoint used in the main comparison. Each batch-size setting used one NVIDIA A100-SXM4-40GB GPU with FP32 tensors; the remaining environment is listed in Section 5.1.3. Each window provides 12 historical flow steps and produces 12-step inflow and outflow forecasts for 1,024 nodes.

For batch size 1, we used compiled BCMS operators and CUDA Graph with 20 warm-up and 100 timed iterations. For batch size 64, we used 10 warm-up and 30 timed iterations. CUDA synchronization preceded and followed each measurement. We report the median and 95th percentile (P95).

Forward-only timing includes model computation on GPU-resident inputs and excludes input copying. End-to-end timing includes memory-resident data access, wavelet decomposition, normalization, state-evidence construction, CPU/GPU transfers, inference, inverse standardization, and returning results to the CPU, as well as input updates before CUDA Graph replay. Both measurements exclude startup preparation, offline graph construction, and cold disk reads.

Table 11 RBSRN inference latency on Chengdu

Inference setting	Batch size	Forward only ms/batch		End to end ms/batch	
		Median	P95	Median	P95
Single-window forecasting	1	15.907	15.930	19.228	19.507
Batch forecasting	64	1138.819	1310.517	1280.394	1464.308

Table 11 reports median forward-only and end-to-end latencies of 15.907 ms and 19.228 ms for a single window, with an end-to-end P95 of 19.507 ms. These measured computation times are below the 5-minute update interval. At batch size 64, the median end-to-end time was 1,280.394 ms/batch and the amortized forward-only time was 17.794 ms/window. Batch size 1 characterizes the response time of an individual request.

The inference configurations above used the same model checkpoint. We compared each against the reference implementation at the same batch size. At batch size 64, checking all 1,723 test windows yielded a maximum absolute prediction difference of 7.63×10^{-6}. Batch size 1 passed a sampled check against the reference at the same batch size. Aggregate forecasting metrics were similar across the two batch sizes, although individual predictions were not identical. All main accuracy results were evaluated at batch size 64.

6 Conclusion

We presented RBSRN, a role-aware and background-conditioned spatiotemporal residual network for short-term multi-step regional inflow and outflow forecasting. RBSRN refines a shared base prediction with state-conditioned expert correction and background-conditioned historical message selection. Across 5–60-minute horizons in Chengdu and Shanghai, it

achieved the lowest AVG12 MAE and RMSE on all four city–channel tasks under positive-target evaluation, with MAE reductions of 0.43%–2.00% relative to the strongest baseline.

Ablation experiments and fixed-model diagnostics separated the roles of SERR and BCMS. At H6, self-evolution correction produced larger absolute MAE benefits for rapidly changing histories, whereas the directed time-lagged benefit varied by city. BCMS produced larger mean benefits for smaller history-derived background discrepancy, and the pattern persisted across the full test set. These results connect the two correction pathways to distinct traffic conditions.

The study uses one month of data from each city and a chronological rolling-origin evaluation protocol. Future work should test temporal generalization across months and seasons, evaluate dynamically updated candidate relations, and compare all methods under both positive-target and all-grid protocols.

References

[1] F. Rodrigues, I. Markou, and F. C. Pereira, "Combining time-series and textual data for taxi demand prediction in event areas: A deep learning approach," *Inf. Fusion*, vol. 49, pp. 120–129, 2019, doi: 10.1016/j.inffus.2018.07.007.

[2] J. Sun, J. Zhang, Q. Li, X. Yi, Y. Liang, and Y. Zheng, "Predicting citywide crowd flows in irregular regions using multi-view graph convolutional networks," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 5, pp. 2348–2359, 2022, doi: 10.1109/TKDE.2020.3008774.

[3] E. Kontou, V. Garikapati, and Y. Hou, "Reducing ridesourcing empty vehicle travel with future travel demand prediction," *Transp. Res. C, Emerg. Technol.*, vol. 121, Art. no. 102826, 2020, doi: 10.1016/j.trc.2020.102826.

[4] J. Ke, H. Zheng, H. Yang, and X. Chen, "Short-term forecasting of passenger demand under on-demand ride services: A spatio-temporal deep learning approach," *Transp. Res. C, Emerg. Technol.*, vol. 85, pp. 591–608, 2017, doi: 10.1016/j.trc.2017.10.016.

[5] A. Essien, I. Petrounias, P. Sampaio, and S. Sampaio, "A deep-learning model for urban traffic flow prediction with traffic events mined from Twitter," *World Wide Web*, vol. 24, no. 4, pp. 1345–1368, 2021, doi: 10.1007/s11280-020-00800-3.

[6] J. Zhang, Y. Zheng, and D. Qi, "Deep spatio-temporal residual networks for citywide crowd flows prediction," in *Proc. AAAI Conf. Artif. Intell.*, vol. 31, no. 1, 2017, pp. 1655–1661, doi: 10.1609/aaai.v31i1.10735.

[7] Y. Li, R. Yu, C. Shahabi, and Y. Liu, "Diffusion convolutional recurrent neural network: Data-driven traffic forecasting," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2018.

[8] B. Yu, H. Yin, and Z. Zhu, "Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting," in *Proc. 27th Int. Joint Conf. Artif. Intell. (IJCAI)*, 2018, pp. 3634–3640, doi: 10.24963/ijcai.2018/505.

[9] Z. Wu, S. Pan, G. Long, J. Jiang, and C. Zhang, "Graph WaveNet for deep spatial-temporal graph modeling," in *Proc. 28th Int. Joint Conf. Artif. Intell. (IJCAI)*, 2019, pp. 1907–1913, doi: 10.24963/ijcai.2019/264.

[10] Y. Fang *et al.*, "When spatio-temporal meet wavelets: Disentangled traffic forecasting via efficient spectral graph attention networks," in *Proc. IEEE 39th Int. Conf. Data Eng. (ICDE)*, 2023, pp. 517–529, doi: 10.1109/ICDE55515.2023.00046.

[11] Y. Xu, L. Wei, and L. Feng, "A spatial-temporal trend-event decoupling dual-channel framework for traffic flow prediction," *Expert Syst. Appl.*, vol. 287, Art. no. 128107, 2025, doi: 10.1016/j.eswa.2025.128107.

[12] X. Zhang *et al.*, "Traffic flow forecasting with spatial-temporal graph diffusion network," in *Proc. AAAI Conf. Artif. Intell.*, vol. 35, no. 17, 2021, pp. 15008–15015, doi: 10.1609/aaai.v35i17.17761.

[13] Q. Long, Z. Fang, C. Fang, C. Chen, P. Wang, and Y. Zhou, "Unveiling delay effects in traffic forecasting: A perspective from spatial-temporal delay differential equations," in *Proc. ACM Web Conf.*, 2024, pp. 1035–1044, doi: 10.1145/3589334.3645688.

[14] Z. Shao *et al.*, "Decoupled dynamic spatial-temporal graph neural network for traffic forecasting," *Proc. VLDB Endow.*, vol. 15, no. 11, pp. 2733–2746, 2022, doi: 10.14778/3551793.3551827.

[15] H. Lee and S. Ko, "TESTAM: A time-enhanced spatio-temporal attention model with mixture of experts," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[16] J. Jiang, C. Han, W. X. Zhao, and J. Wang, "PDFormer: Propagation delay-aware dynamic long-range Transformer for traffic flow prediction," in *Proc. AAAI Conf. Artif. Intell.*, vol. 37, no. 4, 2023, pp. 4365–4373, doi: 10.1609/aaai.v37i4.25556.

[17] R. Jiang *et al.*, "DL-Traff: Survey and benchmark of deep learning models for urban traffic prediction," in *Proc. 30th ACM Int. Conf. Inf. Knowl. Manage. (CIKM)*, 2021, pp. 4515–4525, doi: 10.1145/3459637.3482000.

[18] Z. Shao, Z. Zhang, F. Wang, W. Wei, and Y. Xu, "Spatial-temporal identity: A simple yet effective baseline for multivariate time series forecasting," in *Proc. 31st ACM Int. Conf. Inf. Knowl. Manage. (CIKM)*, 2022, pp. 4454–4458, doi: 10.1145/3511808.3557702.

[19] S. Guo, Y. Lin, N. Feng, C. Song, and H. Wan, "Attention based spatial-temporal graph convolutional networks for traffic flow forecasting," in *Proc. AAAI Conf. Artif. Intell.*, vol. 33, no. 1, 2019, pp. 922–929, doi: 10.1609/aaai.v33i01.3301922.

[20] C. Zheng, X. Fan, C. Wang, and J. Qi, "GMAN: A graph multi-attention network for traffic prediction," in *Proc. AAAI Conf. Artif. Intell.*, vol. 34, no. 1, 2020, pp. 1234–1241, doi: 10.1609/aaai.v34i01.5477.

[21] Z. Wu, S. Pan, G. Long, J. Jiang, X. Chang, and C. Zhang, "Connecting the dots: Multivariate time series forecasting with graph neural networks," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discov. Data Min. (KDD)*, 2020, pp. 753–763, doi: 10.1145/3394486.3403118.

[22] L. Bai, L. Yao, C. Li, X. Wang, and C. Wang, "Adaptive graph convolutional recurrent network for traffic forecasting," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 17804–17815.

[23] C. Song, Y. Lin, S. Guo, and H. Wan, "Spatial-temporal synchronous graph convolutional networks: A new framework for spatial-temporal network data forecasting," in *Proc. AAAI Conf. Artif. Intell.*, vol. 34, no. 1, 2020, pp. 914–921, doi: 10.1609/aaai.v34i01.5438.

[24] M. Li and Z. Zhu, "Spatial-temporal fusion graph neural networks for traffic flow forecasting," in *Proc. AAAI Conf. Artif. Intell.*, vol. 35, no. 5, 2021, pp. 4189–4196, doi: 10.1609/aaai.v35i5.16542.

[25] X. Su, F. Liu, Y. Chang, E. Tanin, M. Sarvi, and J. Qi, "DualCast: A model to disentangle aperiodic events from traffic series," in *Proc. 34th Int. Joint Conf. Artif. Intell. (IJCAI)*, 2025, pp. 3290–3298, doi: 10.24963/ijcai.2025/366.

[26] M. Shao, Z. Zhang, Y. Wang, Y. Dai, X. Shen, and X. Wang, "HyperD: Hybrid periodicity decoupling framework for traffic forecasting," in *Proc. AAAI Conf. Artif. Intell.*, vol. 40, no. 18, 2026, pp. 15689–15697, doi: 10.1609/aaai.v40i18.38599.

[27] J. Ji *et al.*, "Spatio-temporal self-supervised learning for traffic flow prediction," in *Proc. AAAI Conf. Artif. Intell.*, vol. 37, no. 4, 2023, pp. 4356–4364, doi: 10.1609/aaai.v37i4.25555.

[28] M. Li, P. Tong, M. Li, Z. Jin, J. Huang, and X.-S. Hua, "Traffic flow prediction with vehicle trajectories," in *Proc. AAAI Conf. Artif. Intell.*, vol. 35, no. 1, 2021, pp. 294–302, doi: 10.1609/aaai.v35i1.16104.

[29] H. Yao, X. Tang, H. Wei, G. Zheng, and Z. Li, "Revisiting spatial-temporal similarity: A deep learning framework for traffic prediction," in *Proc. AAAI Conf. Artif. Intell.*, vol. 33, no. 1, 2019, pp. 5668–5675, doi: 10.1609/aaai.v33i01.33015668.

[30] R. Jiang *et al.*, "Spatio-temporal meta-graph learning for traffic forecasting," in *Proc. AAAI Conf. Artif. Intell.*, vol. 37, no. 7, 2023, pp. 8078–8086, doi: 10.1609/aaai.v37i7.25976.

[31] W. Ruan, X. Dang, Z. Zhou, S. Lyu, and Y. Liang, "A retrieval augmented spatio-temporal framework for traffic prediction," in *Proc. AAAI Conf. Artif. Intell.*, vol. 40, no. 46, 2026, pp. 39163–39172, doi: 10.1609/aaai.v40i46.41264.

[32] Z. Pan *et al.*, "Spatio-temporal meta learning for urban traffic prediction," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 3, pp. 1462–1476, 2022, doi: 10.1109/TKDE.2020.2995855.

[33] J. Ke, X. Qin, H. Yang, Z. Zheng, Z. Zhu, and J. Ye, "Predicting origin-destination ride-sourcing demand with a spatio-temporal encoder-decoder residual multi-graph convolutional network," *Transp. Res. C, Emerg. Technol.*, vol. 122, Art. no. 102858, 2021, doi: 10.1016/j.trc.2020.102858.

[34] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville, "FiLM: Visual reasoning with a general conditioning layer," in *Proc. AAAI Conf. Artif. Intell.*, vol. 32, no. 1, 2018, pp. 3942–3951, doi: 10.1609/aaai.v32i1.11671.

[35] S. Lan, Y. Ma, W. Huang, W. Wang, H. Yang, and P. Li, "DSTAGNN: Dynamic spatial-temporal aware graph neural network for traffic flow forecasting," in *Proc. 39th Int. Conf. Mach. Learn. (ICML)*, vol. 162, 2022, pp. 11906–11917.

[36] L. Cao, B. Wang, G. Jiang, Y. Yu, and J. Dong, "Spatiotemporal-aware trend-seasonality decomposition network for traffic flow forecasting," in *Proc. AAAI Conf. Artif. Intell.*, vol. 39, no. 11, 2025, pp. 11463–11471, doi: 10.1609/aaai.v39i11.33247.

[37] Y. Zou, H. Yuan, Q. Yang, Z. Yuan, S. Wang, and S. Ruan, "Meta dynamic graph for traffic flow prediction," in *Proc. AAAI Conf. Artif. Intell.*, vol. 40, no. 19, 2026, pp. 16584–16592, doi: 10.1609/aaai.v40i19.38699.

[38] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A time series is worth 64 words: Long-term forecasting with Transformers," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.

[39] S. Wang *et al.*, "TimeMixer: Decomposable multiscale mixing for time series forecasting," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[40] J. Deng, X. Chen, R. Jiang, X. Song, and I. W. Tsang, "ST-Norm: Spatial and temporal normalization for multi-variate time series forecasting," in *Proc. 27th ACM SIGKDD Conf. Knowl. Discov. Data Min. (KDD)*, 2021, pp. 269–278, doi: 10.1145/3447548.3467330.